



Augmentation Examples



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Overview of Techniques

Before exploring each method, see the complete map. The techniques are grouped by objective — the next slide details the common code workflow shared by all of them.

Reference

`ts_aug_none()`

Local Perturbations and Shape

`ts_aug_jitter()`

`ts_aug_flip()`

`ts_aug_stretch()`

`ts_aug_shrink()`

`ts_aug_wormhole()`

Recency

`ts_aug_awareness()`

`ts_aug_awaresmooth()`

📄 **Main reference:** Fawaz, H. I. et al. (2019). Deep learning for time series classification: A review. *Data Mining and Knowledge Discovery*, 33, 917–963.

General Workflow

1. Setup

```
# Packages
library(daltoolbox)
library(tspredit)

# Series (default: tsd)
data(tsd)
sw_size <- 10
xw <- ts_data(tsd$y, sw_size)
```

In examples 07–08, the series is a cosine with noise and outliers injected manually.

2. Augmentation (varies by method)

```
# Instantiate the method
augment <- ts_aug_X() # ← changes per
example

# Fit and transform
augment <- fit(augment, xw)
xa <- transform(augment, xw)
idx <- attr(xa, "idx")
ts_head(xa)
```

Only the `ts_aug_X()` line changes between examples.

3. Visualization (after the method)

```
i <- 1:nrow(xw)
y <- xw[, sw_size]
plot(x = i, y = y, main = "cosine")
lines(x = i, y = y, col = "black")

for (j in 1:nrow(xa)) {
  lines(
    x = (idx[j] - sw_size + 1):idx[j],
    y = xa[j, 1:sw_size],
    col = "green"
  )
}
```

Black line = original series. Green lines = augmented windows overlaid.

01 – No Augmentation (`ts_aug_none`)

`ts_aug_none()` keeps the training windows unchanged, functioning as an identity transform. Its role is methodological: it establishes the reference case (baseline) against which any augmentation gain will be measured. Without this anchor, it is not possible to know whether synthetic windows actually help the predictor.

```
# Augmentation (none)
augment <- ts_aug_none()
augment <- fit(augment, xw)
xa <- transform(augment, xw)
idx <- attr(xa, "idx")
ts_head(xa)
```

Step by step

- Instantiate `ts_aug_none()` — identity, no transformation
- Fit with `fit()` and transform with `transform()`
- Inspect the windows with `ts_head(xa)`

📌 **Key concept:** `ts_aug_none()` keeps the original windows unchanged. Its value is methodological: it provides the **reference case** for judging whether synthetic windows actually help the predictor.

02 – Jitter Augmentation (ts_aug_jitter)

ts_aug_jitter() applies small random perturbations to each sliding window, preserving the overall local pattern. The goal is to make the predictor less sensitive to minor measurement noise, expanding the training set without altering the forecasting flow.

```
# Augmentation (jitter)
augment <- ts_aug_jitter()
augment <- fit(augment, xw)
xa <- transform(augment, xw)
idx <- attr(xa, "idx")
ts_head(xa)
```

Step by step

- Replace ts_aug_none() with ts_aug_jitter()
- Small random perturbations are added to each window
- The overall local pattern is preserved

📌 **Key concept:** Jitter adds small random noise to make the predictor **less sensitive to measurement errors**, without altering the temporal structure of the window. Ref: Um et al. (2017), ACM ICMI.

03 – Flip Augmentation (`ts_aug_flip`)

`ts_aug_flip()` mirrors each window around its mean or central behavior, creating symmetric shape variations. The technique exposes the learner to a wider set of local patterns without introducing stochastic noise — an augmentation based purely on shape.

```
# Augmentation (flip)
augment <- ts_aug_flip()
augment <- fit(augment, xw)
xa <- transform(augment, xw)
idx <- attr(xa, "idx")
ts_head(xa)
```

Step by step

- Replace with `ts_aug_flip()`
- Each window is mirrored around its mean or central behavior
- Generates symmetric variations of local patterns
- The learner sees a wider set of local shapes

 **Key concept:** Flip is a **shape-based** augmentation — it creates local symmetry without introducing stochastic noise. Useful for exposing the model to plausible inverted patterns. Ref: Fawaz et al. (2019).

04 – Stretch Augmentation (ts_aug_stretch)

`ts_aug_stretch()` amplifies the internal variations of each window, simulating stronger local oscillations than those observed in the original sample. The temporal structure is preserved; only the amplitude is expanded, exposing the model to more intense local movements.

```
# Augmentation (stretch)
augment <- ts_aug_stretch()
augment <- fit(augment, xw)
xa      <- transform(augment, xw)
idx     <- attr(xa, "idx")
ts_head(xa)
```

Step by step

- Replace with `ts_aug_stretch()`
- Variations within each window are **amplified**
- Simulates stronger local oscillations than those observed in the original sample
- The temporal structure is maintained; only the amplitude changes

📌 **Key concept:** Stretch exposes the model to **more intense local movements**, creating alternative windows through amplitude expansion. Ref: Fawaz et al. (2019).

05 – Shrink Augmentation (`ts_aug_shrink`)

`ts_aug_shrink()` compresses the internal variations of each window, producing smoother versions of the original pattern. It is the direct complement of Stretch: while Stretch exposes the model to stronger oscillations, Shrink makes it robust to lower-amplitude patterns.

```
# Augmentation (shrink)
augment <- ts_aug_shrink()
augment <- fit(augment, xw)
xa <- transform(augment, xw)
idx <- attr(xa, "idx")
ts_head(xa)
```

Step by step

- Replace with `ts_aug_shrink()`
- The variations within each window are **compressed**
- Produces smoother versions of the original pattern
- Direct complement of Stretch — compare both to evaluate robustness to amplitude

 **Key concept:** Shrink is the opposite of Stretch — it compresses amplitude to make the model robust to **smoother patterns**. Ref: Fawaz et al. (2019).

06 – Wormhole Augmentation (ts_aug_wormhole)

ts_aug_wormhole() replaces selected lags with older values from the same series, creating plausible alternative histories that still respect the temporal structure. It is a structural augmentation: it does not alter amplitude or shape, but recomposes the history with real past values.

```
# Augmentation (wormhole)
augment <- ts_aug_wormhole()
augment <- fit(augment, xw)
xa      <- transform(augment, xw)
idx     <- attr(xa, "idx")
ts_head(xa)
```

Step by step

- Replace with ts_aug_wormhole()
- Some lags are replaced by older values from the same series
- Creates **plausible alternative histories** respecting the temporal nature
- More structural strategy than simple noise injection

 **Key concept:** Wormhole is a **structural** augmentation — it does not alter amplitude, but replaces lags with real past values, creating temporally coherent alternative histories. Ref: Fawaz et al. (2019).

07 – Recency-Aware Augmentation (`ts_aug_awareness`)

 **Note:** this example uses a cosine series with noise and 3 manually injected outliers (positions 30, 60, 90) — see the General Workflow slide for the series variant.

`ts_aug_awareness()` weights training windows by giving greater importance to the most recent ones, encoding temporal preference rather than just geometric variation. It is especially useful when the recent regime of the series differs from the older history — such as in series with outliers or pattern shifts.

```
# Augmentation (awareness)
filter <- ts_aug_awareness(0.25)
xa <- transform(filter, xw)
idx <- attr(xa, "idx")
ts_head(xa)
```

Step by step

- Instantiate `ts_aug_awareness(0.25)` — parameter controls the degree of recency emphasis
- More recent windows receive greater weight in the training set
- Older, less relevant patterns are under-represented

 **Key concept:** Awareness encodes **temporal preference** — useful when the recent regime of the series differs from its history. Ref: Fawaz et al. (2019).

08 – Aware-Smoothing Augmentation (`ts_aug_awaresmooth`)

 **Note:** same series as example 07 — cosine with noise and 3 outliers (positions 30, 60, 90).

`ts_aug_awaresmooth()` combines recency emphasis with local smoothing of windows — a hybrid augmentation. It reduces short-term noise while preserving the structure of the most recent regime. Ideal when the recent period is the most relevant, but still presents significant noise.

```
# Augmentation (awareness + smoothing)
filter <- tspredit::ts_aug_awaresmooth(0.25)
xa <- transform(filter, xw)
idx <- attr(xa, "idx")
ts_head(xa)
```

Step by step

- Replace with `tspredit::ts_aug_awaresmooth(0.25)` — combines recency emphasis with window smoothing
- Reduces short-term noise while preserving the recent structure
- Hybrid augmentation: alters both sample emphasis and local smoothness

 **Key concept:** Aware-Smoothing is a **hybrid** augmentation — combines temporal preference (recency) with local smoothing. Ideal when the recent regime is the most relevant, but still noisy. Ref: Fawaz et al. (2019).

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

